

CIRRUS

Smart Multi-Cloud Storage Pooling and Compression Platform

Project Report

Submitted in partial fulfillment of the requirements of the course

[Course Name and Code]

[Department / Institution Name]

Submitted by

Member 1: [Full Name and Roll Number]

Member 2: [Full Name and Roll Number]

Member 3: [Full Name and Roll Number]

Member 4: [Full Name and Roll Number]

Session 2025 to 2026

Table of Contents

1. Introduction	1
2. System Architecture	2
3. Database Design	3
4. Core Features and Implementation	4
5. Security and Privacy	5
6. User Interface	6
7. Deployment Architecture	7
8. Team Contributions	8
9. Conclusion and Future Work	9

1. Introduction

1.1 Overview

CIRRUS is a web based platform that allows a user to connect several free cloud storage accounts, such as Google Drive and Dropbox, and use them together as a single unified storage space. Instead of treating each cloud account separately, CIRRUS pools the free quota of every connected account into one combined capacity. Every file that a user uploads is first compressed and then automatically routed to the connected cloud that currently has the largest amount of free space.

1.2 Problem Statement

Individual cloud providers offer only a limited amount of free storage, for example fifteen gigabytes on Google Drive and two gigabytes on Dropbox. Users frequently hold several such accounts, yet the free space remains fragmented and difficult to use as a whole. There is no convenient consumer tool that presents these scattered free allocations as one large and easy to manage storage box.

1.3 Objectives

- To let users connect multiple free cloud accounts through a simple and secure sign in process.
- To present the combined free capacity of all connected accounts as one pooled storage box.
- To compress files automatically and route each file to the cloud with the most available space.
- To organize stored files using a unified virtual folder structure independent of the physical cloud location.
- To protect user accounts and cloud credentials using strong cryptographic measures.

1.4 Scope

The platform supports user registration and authentication, connection of Google Drive and Dropbox accounts through the official authorization flow, compression of files, automatic storage routing, virtual folder navigation, bulk uploads, and a dashboard that reports the pooled capacity and storage savings. The system stores only metadata and encrypted credentials, while the actual file contents reside on the user owned cloud accounts.

2. System Architecture

2.1 Architectural Overview

CIRRUS follows a three tier architecture. The presentation tier is a single page web application built with React. The application tier is a backend service built with the FastAPI framework. The data tier is a managed PostgreSQL database hosted on Supabase. The backend communicates with external cloud providers through their official application programming interfaces to transfer the actual file data.

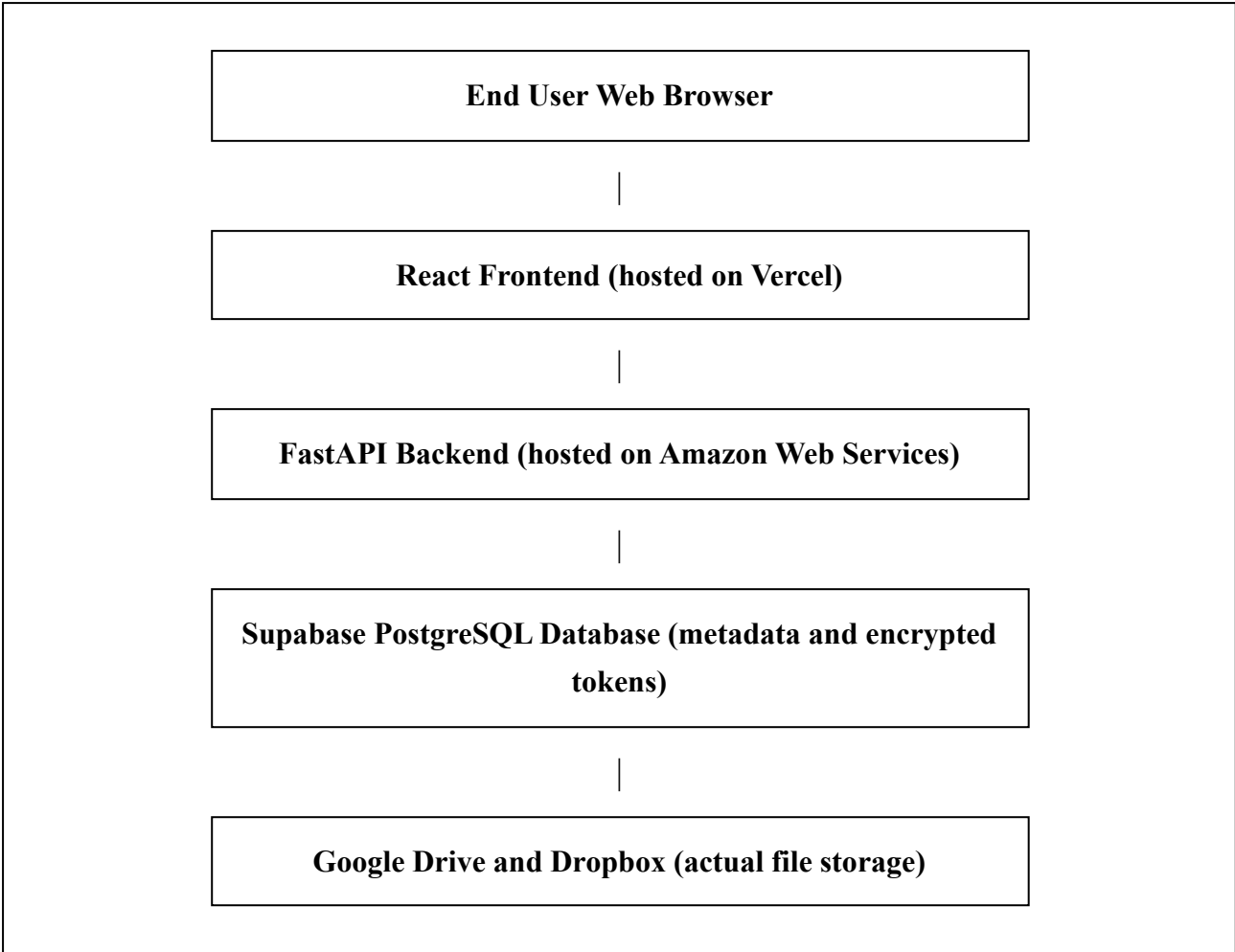


Figure 2.1: High level three tier architecture of the CIRRUS platform.

2.2 Technology Stack

Layer	Technology	Purpose
Frontend	React, Vite, React Router	User interface, routing, and client logic

Backend	Python, FastAPI, Uvicorn	API, authentication, compression, and routing engine
Database	PostgreSQL on Supabase	Storage of users, accounts, folders, files, and logs
Cloud APIs	Google Drive API, Dropbox API	Physical upload, download, and quota reading
Hosting	Amazon Web Services, Vercel	Backend server and frontend delivery

2.3 Data Flow

When a user uploads a file, the browser sends the file to the backend together with the authentication token. The backend compresses the file, queries the live free space of every connected account, selects the account with the most available space, and uploads the compressed file to that cloud through its official interface. The backend then records the file metadata in the database and returns a direct web link to the stored file.

3. Database Design

3.1 Overview

The database consists of five related tables. The design isolates user identity, connected storage accounts, virtual folders, stored file metadata, and an activity audit trail. Each record uses a universally unique identifier as its primary key. Only metadata and encrypted credentials are persisted, and no file content is stored in the database.

3.2 Tables

Table	Key Columns	Description
users	id, email, hashed_password, full_name, created_at	Registered user accounts with securely hashed passwords
storage_accounts	id, user_id, provider, display_name, credentials_json, quota_limit, used_space	Connected cloud accounts with encrypted refresh tokens and live quota figures
folders	id, user_id, name, parent_id, created_at	Virtual folder tree using a self reference for nesting
stored_files	id, user_id, folder_id, account_id, original_name, compression_type, original_size, compressed_size, web_link	Metadata for every uploaded file and its physical cloud location
audit_logs	id, user_id, action, details, timestamp	Record of user actions such as sign up, login, upload, and delete

3.3 Relationships

- A user owns many storage accounts, folders, files, and audit log entries.
- A storage account contains many stored files.
- A folder contains many files and may contain child folders through the parent identifier.

4. Core Features and Implementation

4.1 Storage Pooling

The platform aggregates the free capacity of every connected account. The free space of an account is computed as the quota limit minus the used space, and the dashboard reports the sum across all accounts as a single pooled figure.

4.2 Compression and Smart Routing

Each uploaded file is compressed using the gzip or zip algorithm before transfer. The routing engine then selects the connected account with the greatest free space and uploads the compressed file to that account. This approach balances usage across accounts and maximizes the effective capacity of the pool.

4.3 Virtual Folders and Bulk Upload

Files are organized through a unified virtual folder tree maintained in the database. A single folder may contain files that physically reside on different clouds, which preserves the concept of one combined storage box. The file manager supports selecting and uploading many files at once, and each file in a batch is routed independently.

4.4 Activity Dashboard

The dashboard summarizes the total number of files, the combined usage and limit, the space saved through compression, and a recent activity log. It provides the user with a clear and immediate understanding of the state of the pooled storage.

5. Security and Privacy

5.1 Authentication

User accounts are protected with passwords that are hashed using a salted PBKDF2 function based on the SHA256 algorithm. Sessions are maintained using signed JSON Web Tokens that expire after a fixed period.

5.2 Credential Encryption

Cloud authorization is performed using the official authorization code flow, which means the platform never receives or stores user passwords for the connected clouds. The resulting refresh tokens are encrypted at rest using symmetric Fernet encryption before they are written to the database.

5.3 Audit Logging

Significant actions, including registration, login, upload, and deletion, are recorded in an audit log table associated with the user. This provides traceability and supports accountability within the system.

6. User Interface

The interface uses a clean red and white visual theme. The following figures present the principal screens of the platform.

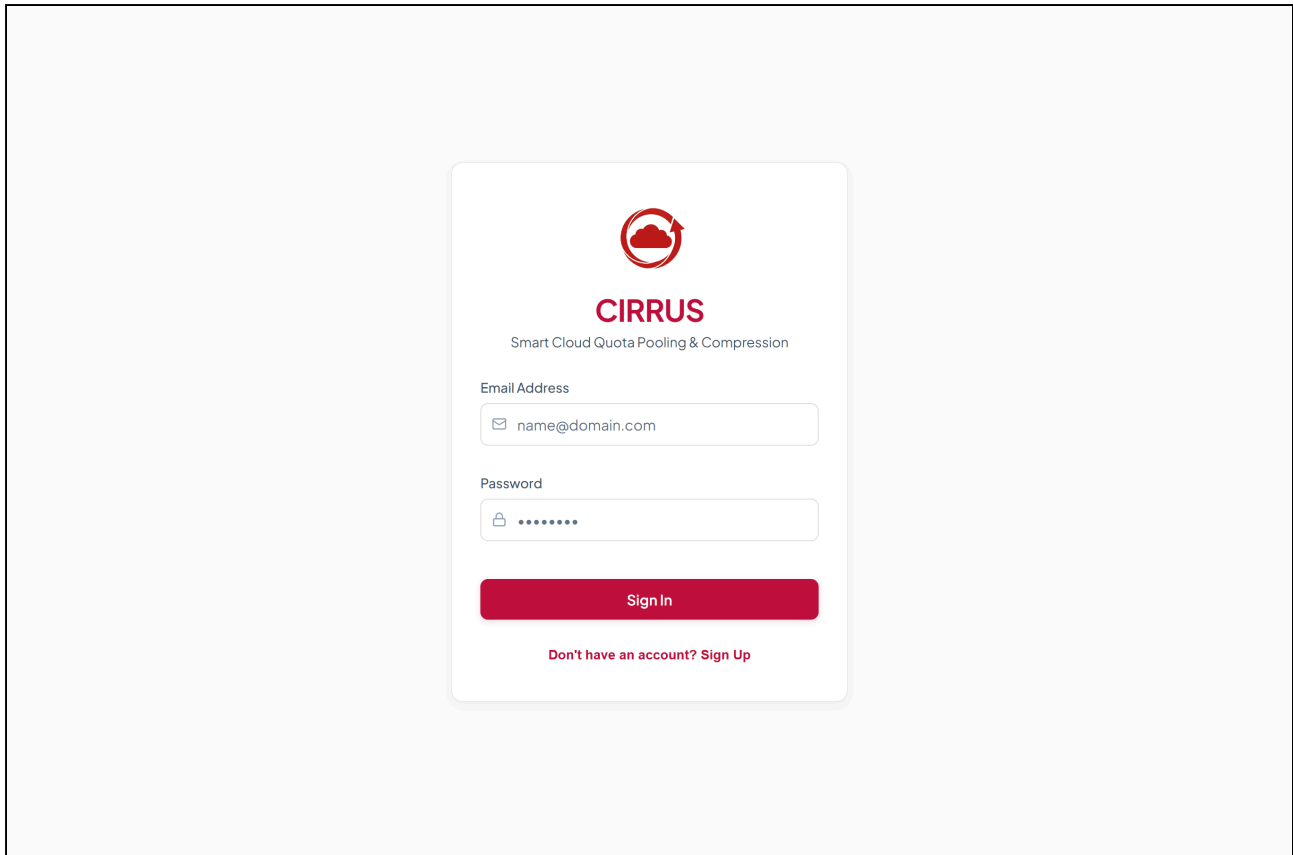


Figure 6.1: Sign in screen for returning users.



CIRRUS

Smart Cloud Quota Pooling & Compression

Full Name

Email Address

Password

☐ I agree to the [Terms of Service](#) & [Privacy Policy](#)

Create Account

Already have an account? [Sign In](#)

Figure 6.2: Registration screen with agreement to the terms of service and privacy policy.

**CIRRUS**

Dashboard

Cloud Connections

File Manager

Good morning, Demo User

Smart, compressed, multi-cloud storage allocation

Refresh Stats

Total Registered Files ⓘ
0
Managed across cloud networks

Combined Usage ⓘ
0 Bytes
Of 0 Bytes aggregate limit

Compression Space Saved ⓘ
0 Bytes -0%
Original: 0 Bytes

Aggregate Space Allocation ⓘ

Storage Used 0% (0 Bytes / 0 Bytes)

SMART ROUTING ACTIVE

Files are automatically routed to the cloud drive with the largest available free space after undergoing high-ratio gzip/zip compression.

Attached Cloud Allocations

No storage accounts registered. Link a drive to start uploading.

Recent Activity Logs ⓘ

SIGNUP

05:22:20 AM

User registered: demo1781846519@cirrus.app

demo1781846519@cirrus.app

Sign Out

Figure 6.3: Dashboard showing pooled usage, compression savings, and recent activity.

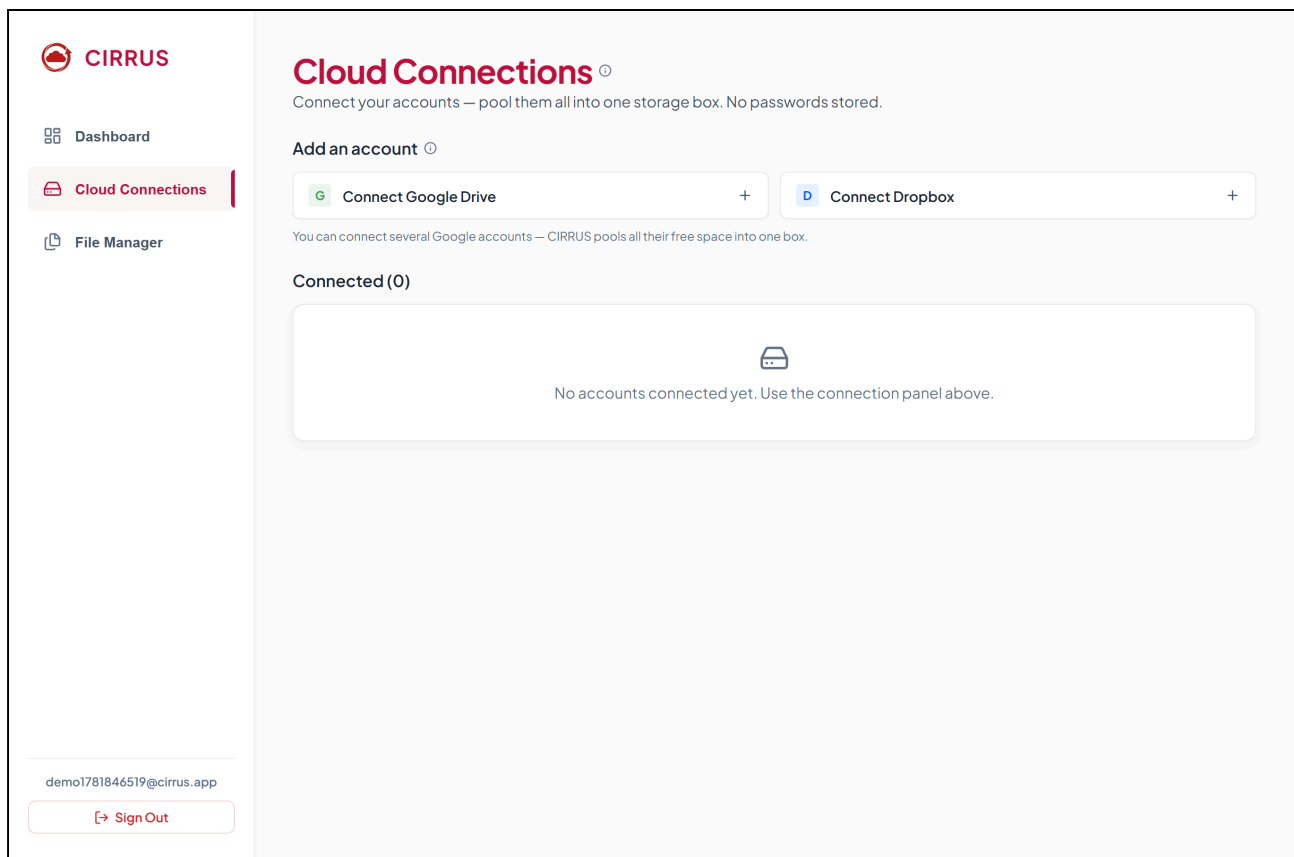


Figure 6.4: Cloud connections screen for linking Google Drive and Dropbox accounts.

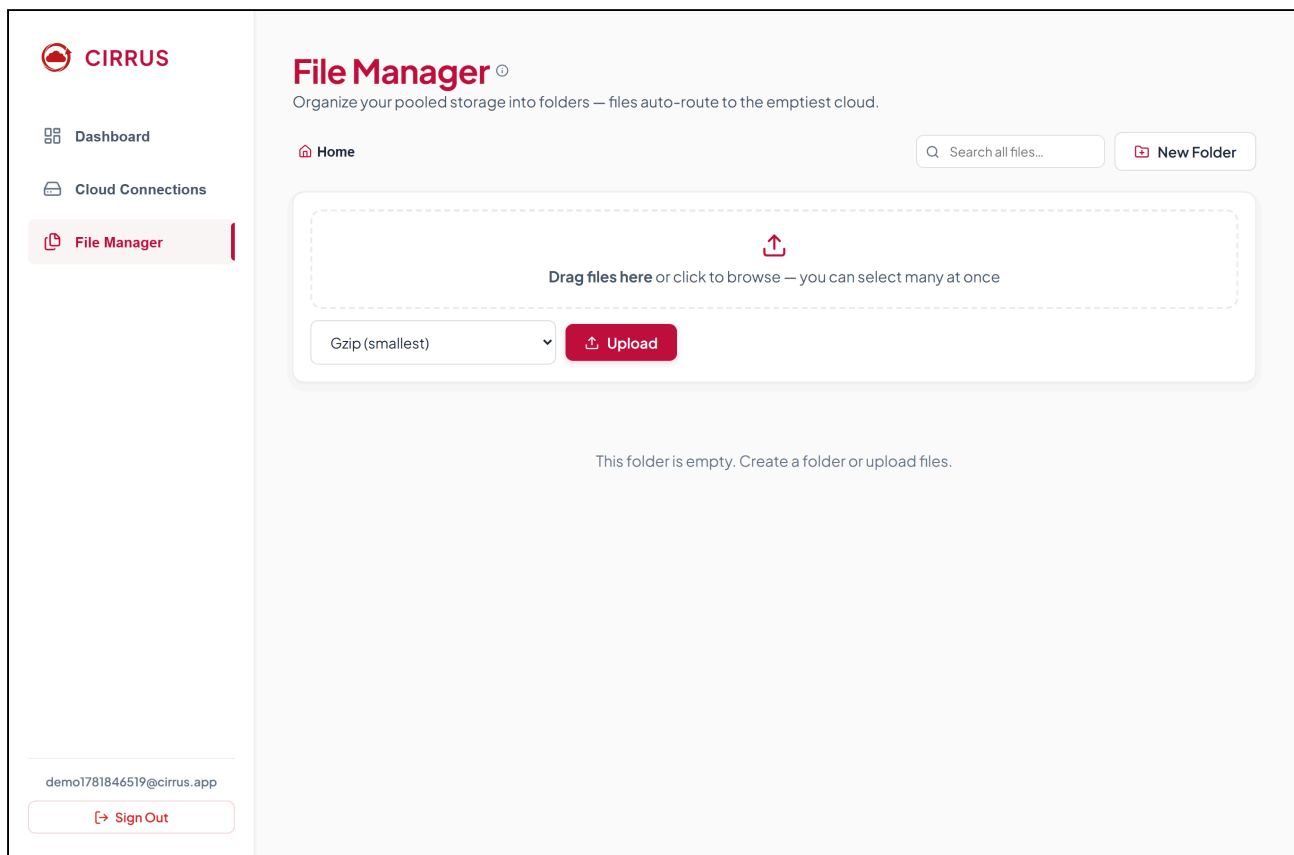


Figure 6.5: File manager with folder navigation, search, and bulk upload.

7. Deployment Architecture

The application is deployed as a live service. The frontend is built and served through Vercel. The backend runs on an Amazon Web Services instance behind a reverse proxy with a trusted certificate, which is required for secure authorization callbacks. The database is the managed PostgreSQL service provided by Supabase. Application secrets, including the database connection string and signing keys, are stored as protected environment variables and are never committed to the source repository.

8. Team Contributions

The project was completed by a team of four members. The responsibilities were divided into four distinct layers so that the workload was balanced and the areas of ownership did not overlap.

8.1 Member 1, Backend Core and Database

The first member was responsible for the backend service and the data layer. This member designed the relational schema of five tables, configured the PostgreSQL database on Supabase, and implemented the database migration logic. The member also developed the core application programming interface for files, folders, and dashboard statistics, and built the compression pipeline together with the smart routing engine that selects the emptiest cloud for each upload.

8.2 Member 2, Authentication and Security

The second member was responsible for all matters of authentication and security. This member implemented user registration and login, the salted password hashing scheme, and the signed token based session system. The member also implemented the symmetric encryption of cloud credentials at rest and developed the audit logging mechanism that records significant user actions.

8.3 Member 3, Cloud Integration

The third member was responsible for the integration with external cloud providers. This member implemented the official authorization code flow for Google Drive and Dropbox, including the secure handling and refreshing of access tokens. The member also developed the storage client layer that performs upload, download, deletion, live quota reading, and the generation of direct web links to stored files.

8.4 Member 4, Frontend and Deployment

The fourth member was responsible for the web application and its delivery. This member built the React interface, including the dashboard, the cloud connections manager, and the file manager with folder navigation and bulk upload. The member also designed the red and white visual theme and the guidance tooltips, and carried out the deployment of the frontend, backend, and database to their respective hosting platforms.

9. Conclusion and Future Work

9.1 Conclusion

CIRRUS successfully demonstrates that fragmented free cloud allocations can be unified into a single and practical storage box. The platform combines secure authentication, encrypted credential handling, file compression, intelligent routing, and a clear user interface into a complete and deployed product.

9.2 Future Work

- Adding support for further providers such as OneDrive once organizational restrictions are resolved.
- Implementing the splitting of very large files across multiple clouds with automatic reassembly on download.
- Introducing file sharing between users and richer search and preview capabilities.